End-to-End Time Series Forecasting System with API

Project Overview

This project builds a production-ready time series forecasting system that:

- Trains multiple forecasting algorithms
- Compares model performance automatically
- Selects the best-performing model
- Forecasts the next 8 weeks of sales for each state
- Exposes predictions using a REST API

Implemented Models: 1. ARIMA / SARIMA 2. Facebook Prophet 3. XGBoost 4. LSTM

Tech Stack:

- Python
- Pandas / NumPy
- Scikit-learn
- XGBoost
- Prophet
- TensorFlow / Keras
- FastAPI
- Matplotlib / Seaborn

---

1. Project Folder Structure

```
forecasting-system/
│
├── data/
│   └── sales_data.xlsx
│
├── notebooks/
│   └── eda.ipynb
│
├── models/
│   ├── best_model.pkl
│   ├── xgboost_model.pkl
│   └── lstm_model.h5
│
```

```
├── src/
│   ├── preprocess.py
│   ├── feature_engineering.py
│   ├── train_models.py
│   ├── evaluate.py
│   ├── predict.py
│   └── api.py
│
├── requirements.txt
├── README.md
└── main.py
```

---

2. Install Required Libraries

Create a virtual environment:

```
python -m venv venv
```

Activate environment:

Windows:

```
venv\Scripts\activate
```

Install dependencies:

```
pip install pandas numpy matplotlib seaborn scikit-learn xgboost prophet
tensorflow statsmodels fastapi uvicorn openpyxl holidays joblib
```

---

3. Load Dataset

preprocess.py

```
import pandas as pd


def load_data(file_path):
```

```
    df = pd.read_excel(file_path)

    print(df.head())
    print(df.info())

    return df
```

Explanation:

- Reads Excel dataset
- Displays structure of dataset
- Helps verify columns and datatypes

---

4. Data Cleaning

preprocess.py

```
import pandas as pd


def clean_data(df):
    # Remove duplicates
    df = df.drop_duplicates()

    # Convert date column
    df['Date'] = pd.to_datetime(df['Date'])

    # Sort values
    df = df.sort_values('Date')

    # Handle missing values
    df['Sales'] = df['Sales'].fillna(method='ffill')

    return df
```

Explanation:

- Removes duplicate records
- Converts Date column into datetime format
- Sorts data chronologically
- Fills missing sales values using forward fill

---

5. Exploratory Data Analysis (EDA)

notebooks/eda.ipynb

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,6))
plt.plot(df['Date'], df['Sales'])
plt.title('Sales Trend')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.show()
```

Check:

- Trend
- Seasonality
- Missing dates
- Outliers

---

6. Feature Engineering

feature_engineering.py

```
import holidays


def create_features(df):

    # Lag features
    df['lag_1'] = df['Sales'].shift(1)
    df['lag_7'] = df['Sales'].shift(7)
    df['lag_30'] = df['Sales'].shift(30)

    # Rolling statistics
    df['rolling_mean_7'] = df['Sales'].rolling(window=7).mean()
    df['rolling_std_7'] = df['Sales'].rolling(window=7).std()

    # Date features
    df['day_of_week'] = df['Date'].dt.dayofweek
    df['month'] = df['Date'].dt.month
```

```
    df['week'] = df['Date'].dt.isocalendar().week

    # Holiday feature
    india_holidays = holidays.India()
    df['holiday'] = df['Date'].isin(india_holidays)

    # Remove nulls created by lagging
    df = df.dropna()

    return df
```

Explanation:

Lag Features:

- Previous sales values
- Helps model understand historical behavior

Rolling Mean / Std:

- Captures trends and volatility

Date Features:

- Captures seasonality patterns

Holiday Flag:

- Helps identify holiday-related sales spikes

---

7. Train-Test Split

train_models.py

```
def train_test_split_time_series(df):

    train = df[:-56]
    test = df[-56:]

    return train, test
```

Explanation:

- Last 56 days used for testing

- Prevents data leakage
- Correct approach for time series forecasting

---

8. ARIMA / SARIMA Model

train_models.py

```
from statsmodels.tsa.statespace.sarimax import SARIMAX


def train_sarima(train):

    model = SARIMAX(
        train['Sales'],
        order=(1,1,1),
        seasonal_order=(1,1,1,7)
    )

    model_fit = model.fit(disp=False)

    return model_fit
```

Explanation:

- Captures trend + seasonality
- Seasonal cycle set to 7 days

---

9. Prophet Model

train_models.py

```
from prophet import Prophet


def train_prophet(train):

    prophet_df = train[['Date', 'Sales']]
    prophet_df.columns = ['ds', 'y']

    model = Prophet()
    model.fit(prophet_df)
```

```
    return model
```

Explanation:

- Automatically handles trend and seasonality
- Strong for business forecasting tasks

---

10. XGBoost Model

train_models.py

```
from xgboost import XGBRegressor


def train_xgboost(train, features):

    X_train = train[features]
    y_train = train['Sales']

    model = XGBRegressor(
        n_estimators=100,
        learning_rate=0.05,
        max_depth=5
    )

    model.fit(X_train, y_train)

    return model
```

Explanation:

- Uses engineered features
- Handles nonlinear patterns effectively

---

11. LSTM Model

train_models.py

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense
from sklearn.preprocessing import MinMaxScaler
import numpy as np


def train_lstm(train):

    scaler = MinMaxScaler()

    scaled_data = scaler.fit_transform(train[['Sales']])

    X_train = []
    y_train = []

    sequence_length = 30

    for i in range(sequence_length, len(scaled_data)):
        X_train.append(scaled_data[i-sequence_length:i, 0])
        y_train.append(scaled_data[i, 0])

    X_train = np.array(X_train)
    y_train = np.array(y_train)

    X_train = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))

    model = Sequential()

    model.add(LSTM(64, activation='relu', input_shape=(X_train.shape[1], 1)))
    model.add(Dense(1))

    model.compile(optimizer='adam', loss='mse')

    model.fit(X_train, y_train, epochs=20, batch_size=16)

    return model
```

Explanation:

- Deep learning approach for sequential forecasting
- Learns temporal dependencies

12. Model Evaluation

evaluate.py

```
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
import numpy as np


def evaluate_model(actual, predicted):

    mae = mean_absolute_error(actual, predicted)

    rmse = np.sqrt(mean_squared_error(actual, predicted))

    mape = np.mean(np.abs((actual - predicted) / actual)) * 100

    return {
        'MAE': mae,
        'RMSE': rmse,
        'MAPE': mape
    }
```

Explanation:

MAE:

- Average prediction error

RMSE:

- Penalizes large errors

MAPE:

- Percentage forecasting accuracy

---

13. Automatic Best Model Selection

main.py

```
results = {
    'sarima': 210.5,
    'prophet': 190.2,
    'xgboost': 150.8,
    'lstm': 170.3
}

best_model = min(results, key=results.get)

print('Best Model:', best_model)
```

Explanation:

- Automatically selects model with lowest RMSE

---

14. Save Best Model

predict.py

```
import joblib

joblib.dump(model, 'models/best_model.pkl')
```

Load model:

```
model = joblib.load('models/best_model.pkl')
```

---

15. Build REST API

api.py

```
from fastapi import FastAPI
import joblib
```

```
app = FastAPI()

model = joblib.load('models/best_model.pkl')


@app.get('/')
def home():
    return {'message': 'Forecast API Running'}


@app.get('/predict/{state}')
def predict(state: str):

    # Example prediction
    forecast = [1000, 1050, 1100]

    return {
        'state': state,
        'forecast': forecast
    }
```

Explanation:

- REST API exposes forecasting results
- Can be consumed by frontend or external systems

---

16. Run API

```
uvicorn src.api:app --reload
```

Open:

```
http://127.0.0.1:8000/docs
```

FastAPI automatically generates Swagger UI.

---

17. Recommended Visualizations

Create:

1. Actual vs Predicted Sales
2. State-wise Forecast Trend
3. RMSE Comparison Chart
4. Feature Importance Plot (XGBoost)

Example:

```
plt.plot(actual)
plt.plot(predicted)
plt.legend(['Actual', 'Predicted'])
plt.show()
```

---

18. Deployment (Optional)

You can deploy API using:

- Render
- Railway
- AWS EC2
- Azure App Service

---

19. Recommended README Structure

Include:

- Project overview
- Architecture diagram
- Dataset information
- Feature engineering steps
- Model comparison table
- API screenshots
- Swagger UI screenshots
- Setup instructions
- Future improvements

---

20. Architecture Flow

```
Dataset
   ↓
Data Cleaning
   ↓
Feature Engineering
   ↓
Train Multiple Models
   ↓
Evaluate Models
   ↓
Select Best Model
   ↓
Save Model
   ↓
REST API
   ↓
Forecast Output
```

---

21. Final Resume Bullet Points

```
• Built an end-to-end time series forecasting system using ARIMA, Prophet, XGBoost, and LSTM models

• Engineered lag, rolling statistics, and seasonal features improving forecasting performance

• Implemented automated model comparison and selection using RMSE and MAE metrics

• Developed REST APIs using FastAPI to serve real-time sales forecasts

• Forecasted next 8 weeks of sales across multiple states using historical sales data
```

---

22. Interview Questions You Should Prepare

1. Why use time-series split instead of random split?

2. Difference between ARIMA and SARIMA?
3. Why use lag features?
4. Why does XGBoost work for forecasting?
5. What is stationarity?
6. Why use rolling mean/std?
7. How does LSTM capture temporal dependencies?
8. Why use RMSE instead of accuracy?
9. How does Prophet handle seasonality?
10. Explain your API architecture.

---

23. Final Project Outcome

By completing this project, you demonstrate:

- Time Series Forecasting
- Machine Learning
- Deep Learning
- Feature Engineering
- Model Evaluation
- Backend API Development
- Production-style ML Pipeline
- Deployment Concepts

This is a strong project for:

- ML Engineer roles
- Data Scientist roles
- Forecasting/Analytics roles
- Backend ML roles